

A self-supervised learning approach to 3D object classification with contrastive learning

Alessandro Borgherini[†]

Abstract—This project investigates 3D object classification using a self-supervised contrastive learning framework based on the ORION encoder architecture. ORION, originally designed to enhance pose-awareness through auxiliary orientation estimation, is repurposed here to generate discriminative embeddings via the NT-Xent loss. The training pipeline consists of two stages: an unsupervised contrastive pretraining phase using data augmentations and a supervised fine-tuning phase for downstream classification. Our experiments on the ModelNet10 dataset evaluate the performance of several architectural variants, showing that while contrastive pretraining helps structure the representation space, it can suffer from overfitting when paired with deeper encoders. Nonetheless, the modular design of the framework allows efficient transfer learning by reusing the encoder across tasks, with potential reductions in training time. Results highlight the trade-offs between encoder size, loss dynamics, and classification accuracy, suggesting avenues for further optimization.

Index Terms—Unsupervised Learning, 3D classification, Optimization, Neural Networks, Convolutional Neural Networks.

I. INTRODUCTION

In recent years, the use of devices capable of generating 3D point clouds has become increasingly widespread. Examples include range sensors integrated into autonomous vehicles and robots, as well as depth cameras such as the Microsoft Kinect. Additionally, methods like Structure from Motion (SfM) and Simultaneous Localization and Mapping (SLAM) have matured significantly, enabling the production of accurate and dense point clouds. With the advent and rapid progress of deep learning, the traditional practice of manually crafting features for recognition tasks has largely been replaced by data-driven approaches, where neural networks automatically learn discriminative representations directly from raw 3D data. [1] [2]

In this project, we investigate classification of 3D objects using a contrastive learning framework built upon Orientation-boosted VoxNet (often called ORION) [3] as a baseline encoder.

ORION improves upon the original VoxNet architecture by adding object orientation estimation as an auxiliary task alongside object class prediction. This multi-task training strategy has been shown to significantly improve classification accuracy for 3D object recognition across datasets like ModelNet10 and ModelNet40, by making features more discriminative with respect to object pose as well as class label.

Our framework adopts a contrastive learning paradigm, where pairs of augmented views of the same object (positives) are pulled together in the embedding space, while views of different objects (negatives) are pushed apart typically via the Normalized Temperature-scaled Cross Entropy (NT-Xent) objective. This allows the model to learn robust, discriminative representations useful for downstream 3D classification. [4]

In our approach, we adopt ORION as the encoder backbone within a contrastive learning pipeline. Specifically, we first apply a projection head and train with the NT-Xent loss, aiming to learn pose-aware, discriminative embeddings. After this stage, we attach a classification head to finetune the embedding space for the ultimate downstream task of object classification.

Related works:

- Nazir et al. proposed a contrastive Auto-Encoder optimized with triplet loss and Chamfer distance to learn both global and local features; the learned representations feed into downstream classifiers like PointNet, yielding improved shape completion and classification accuracy on ModelNet benchmarks [5]
- Iang et al. (2021) introduced an encoder-based contrastive framework using handcrafted transformations for positive sampling. Applying NT-Xent style loss, they showed strong performance on ModelNet10/40 and segmentation tasks without label supervision [6]

II. METHOD

A. Orientation-Boosted VoxNet (ORION) Architecture

Our encoder is based on **ORION** which extends the standard *VoxNet* architecture by incorporating an auxiliary **object orientation estimation** task in parallel with class prediction. ORION is trained in a **multi-task** manner, forcing the network to predict both class label and quantized orientation, thereby encouraging pose-aware internal features that generalize better across rotations.

Network layers and structure:

- **Input representation:** Objects are voxelized into a fixed 3D occupancy grid (in our case $28 \times 28 \times 28$), representing object volume.
- **Convolutional backbone:** Two 3D convolutional layers with increasing channel sizes ($1 \rightarrow 32 \rightarrow 64$) filters of size $3 \times 3 \times 3$. Pooling is applied after the second layer with a $2 \times 2 \times 2$ stride. Each conv layer uses batch normalization and dropout (ratios increasing from ~ 0.2 to ~ 0.3).

[†]Department of Information Engineering, University of Padova, email: alessandro.borgherini@studenti.unipd.it

- **Fully connected layers:** Following the convolutional backbone, a fully connected (FC) layer with 128 units is used, followed by two parallel heads:
 - **FC-class:** outputs class logits (e.g., 10 classes for ModelNet10).
 - **FC-orientation:** outputs quantized orientation bins.

The training objective combines class loss L_C and orientation loss L_O in a weighted sum:

$$L_{\text{total}} = \lambda_C L_C + \lambda_O L_O,$$

often with equal weights. This auxiliary orientation supervision makes filters more sensitive to pose-specific features, leading to improved classification performance.

The full ORION architecture is represented in figure 1

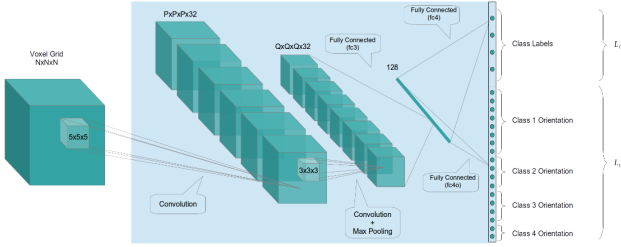


Fig. 1: Model architecture for the orientation boosted Voxnet

B. ORION as the Encoder in a Contrastive Learning Framework

In our proposed method we adopt the pre-trained ORION network (excluding its final heads) as the **encoder backbone** in a self-supervised **contrastive learning** framework. The key components are as follows:

- 1) **Encoder:** All convolutional layers and the first FC layer (128-dim) from ORION are reused as the encoder $f(\cdot)$, providing pose-aware embeddings.
- 2) **Projection (Transform) Head:** A new MLP $g(\cdot)$, typically two FC layers with ReLU activation, projects the encoder output into a contrastive embedding space (128 dimensional vector).
- 3) **Contrastive Pre-training:** We adopt the **Normalized Temperature-scaled Cross Entropy (NT-Xent)** loss [4]. Given an anchor voxel input x and its augmented view x^+ as a positive pair, all other samples in the batch act as negatives:

$$L_{\text{NT-Xent}} = -\log \frac{\exp(\text{sim}(z, z^+)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\text{sim}(z, z_k)/\tau)},$$

where $z = g(f(x))$, τ is the temperature parameter, and $\text{sim}(\cdot, \cdot)$ is the cosine similarity.

- 4) **Fine-tuning Classification Head:** After contrastive training, we attach a new classification head (another 1 hidden layer MLP) to the encoder and train it with cross-entropy loss. The encoder is frozen during the training.

C. Implementation Details

- **Data augmentation:** Random rotations, translations, and voxel jittering generate positive pairs for contrastive training.
- **Batch size and optimization:** Large batches (800 in our case) ensure sufficient negative samples. We use AdamW [7] optimizer with tuned learning rate schedules. In particular we used the cosine annealing learning rate scheduler [8].
- **Training schedule:**

- 1) Stage 1: Contrastive pre-training for a maximum of 3000 epochs using NT-Xent.
- 2) Stage 2: Supervised training of the classification head with cross-entropy.

In both training early stopping is applied after 150 epochs that the validation loss didn't improve.

III. SIGNALS AND FEATURES

This work addresses a 3D object classification task using deep learning techniques. The input signals are 3D mesh models, which are first converted into voxel occupancy grids and subsequently processed through a 3D convolutional neural network encoder based on the convolutional part of the Orion VoxNet architecture.

A. Measurement Setup and Input Data Format

The dataset used for this study is ModelNet10 [9], a widely adopted benchmark in 3D object recognition. It consists of CAD models belonging to 10 object categories, each represented by several 3D triangular meshes stored in the .off (Object File Format) file format. All meshes in the dataset are aligned in a canonical orientation and normalized within a unit bounding box.

Each sample initially contains two components:

- A list of vertices defining the 3D coordinates of the mesh points.
- A list of faces, each face being a triplet of indices into the vertex list, forming triangular surfaces.

B. Signal Pre-processing

To convert the mesh data into a format compatible with 3D convolutional neural networks, we employed a custom transformation that converts each triangular mesh into a binary voxel grid of fixed dimensions $28 \times 28 \times 28$. This transformation is implemented in the ToVoxelGrid class, which operates as follows:

- 1) The triangular mesh is reconstructed using the `trimesh` library and then normalized to fit within the unit cube $[0, 1]^3$. This involves translating the mesh such that its minimum bounding box corner is at the origin and uniformly scaling it so that its maximum extent along any axis equals one.
- 2) The normalized mesh is voxelized using a voxel pitch of $1/28$, resulting in a binary occupancy matrix where each voxel is marked as occupied or empty.

- 3) The resulting voxel grid is then either cropped or zero-padded to ensure a fixed size of $28 \times 28 \times 28$. This step guarantees that all input samples share a consistent shape and resolution.
- 4) Finally, the voxel grid is stored as a tensor in the sample dictionary, replacing the original mesh data.

C. Feature Extraction and Input Representation

The pre-processed input signals are fed into the convolutional encoder of the Orion VoxNet architecture. As such, no handcrafted features are extracted explicitly. Instead, the 3D convolutional layers in the encoder automatically learn hierarchical spatial features directly from the voxel grids. The encoder processes each $28 \times 28 \times 28$ binary tensor as a volumetric image, allowing the network to learn features relevant to object shape and structure.

To accelerate training, all $28 \times 28 \times 28$ voxel tensors are precomputed and saved as `.torch` files, allowing them to be loaded directly during training rather than recomputing the transformation for each sample on the fly.

D. Dataset Splitting

To train and evaluate the model, the dataset was split into two disjoint subsets:

- **Training set:** used to optimize the network parameters.
- **Validation set:** used for model testing.

The specific split ratios and sampling strategies follow standard practice for the ModelNet10 dataset.

E. Data Augmentation

To improve the generalization capability of the model and support contrastive learning (CL), we applied a dedicated data augmentation pipeline on the voxelized input data. The augmentations are designed to simulate realistic geometric variations while preserving the semantic identity of the objects.

a) Cropping and Resizing.: Each time a 3D voxel grid of size $28 \times 28 \times 28$ is sampled, we first apply a random crop to extract a $21 \times 21 \times 21$ subvolume. This crop is taken from a random location within the original grid, ensuring spatial diversity in the object representation. The cropped subvolume is then resized back to the original resolution of $28 \times 28 \times 28$ using nearest-neighbor interpolation. This transformation is implemented in the `RandomCropResize3D` class, which performs both cropping and resizing while preserving binary occupancy values.

b) Rotation.: To generate positive pairs for contrastive learning, we create two distinct augmented versions of each cropped voxel grid. For each version, we apply a random rotation around one of the three principal axes (x , y , or z). The rotation angle is chosen randomly from a fixed set of eight possible values, uniformly spaced between 0 and 2π . The rotation is implemented using 3D affine transformations via PyTorch's `grid_sample` and `affine_grid` functions, and is encapsulated in the `RandomRotate` class.

Together, the cropping and rotation steps serve to introduce geometric variation while retaining semantic consistency, which is critical for learning robust and invariant representations in the contrastive learning framework.

IV. LEARNING FRAMEWORK

The proposed learning framework is composed of two main phases: an initial contrastive learning stage, followed by a supervised classification stage. The same encoder is used in both phases but trained differently, depending on the objective.

A. Encoder Network

As the encoder, we adopt the convolutional backbone of the Orion VoxNet architecture, which processes $28 \times 28 \times 28$ voxel occupancy grids. To obtain a compact feature representation, we append a fully connected linear layer with dropout regularization after the final convolutional layer. This produces a 128-dimensional latent representation vector for each input.

B. Contrastive Learning Phase

In the first training phase, we adopt a contrastive learning approach using the positive sample pairs generated via the data augmentation process described previously. The architecture during this phase is structured as follows:

- The encoder produces a 128-dimensional representation vector.
- This vector is passed through a projection head, implemented as a Multi-Layer Perceptron (MLP) with:
 - One hidden layer of size 1024
 - ReLU activation
 - Output layer of size 128
- The projection head's output is used to compute the contrastive loss.

We use the NT-Xent loss (Normalized Temperature-Scaled Cross Entropy Loss), which encourages representations of positive pairs to be similar while pushing apart those of different instances.

C. Supervised Classification Phase

Once the contrastive learning phase is complete, we freeze the encoder weights and train a classifier on top of the learned representations. The classifier is also a simple MLP consisting of:

- One hidden layer of size 128
- Dropout applied with probability 0.3
- ReLU activation
- Output layer of size 10 (corresponding to the number of classes in ModelNet10)

The classification task is trained using the standard Cross Entropy Loss.

D. Hyperparameters

The table below summarizes all the key hyperparameters used during both training phases:

Hyperparameter	Value
Voxel grid size	$28 \times 28 \times 28$
Crop size	$21 \times 21 \times 21$
Encoder output dimension	128
Projection head hidden size	1024
Projection head output size	128
Classifier hidden size	128
Classifier output size	10
Contrastive loss	NT-Xent
Classification loss	Cross Entropy
Batch size (contrastive learning)	800
Batch size (classification)	300
Optimizer	AdamW
Initial learning rate (contrastive)	1×10^{-4}
Learning rate (classification)	5×10^{-3}
Dropout (encoder FC layer)	0.3
Temperature (NT-Xent)	0.07
Number of orientation bins (K)	8

TABLE 1: Summary of hyperparameters used during training.

V. RESULTS

In this section, we present the results of our experiments, structured progressively to highlight the behavior and performance of the proposed learning framework. The results are evaluated in terms of classification accuracy, precision, recall, and F1-score on the ModelNet10 dataset. We also analyze the effects of different architectural choices and key hyperparameters.

A. Contrastive Learning Performance

We first report the training behavior of the contrastive learning phase. In Figure 2, we show the NT-Xent loss as a function of training epochs. The loss steadily decreases, indicating that the encoder learns to bring positive pairs closer in the representation space.

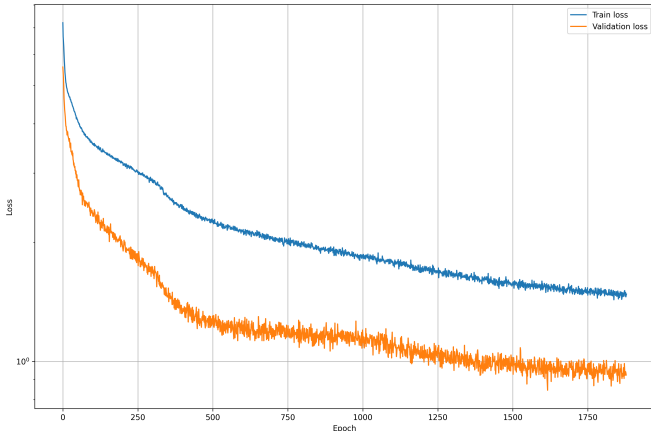


Fig. 2: NT-Xent loss over epochs during contrastive learning.

B. Classification Accuracy

After pretraining the encoder via contrastive learning, we train a classifier on top of the frozen encoder. Table 2

summarizes the performance in terms of accuracy, precision, recall, and F1-score across all 10 object categories.

Class	Precision	Recall	F1-score
Bathtub	0.92	0.70	0.80
Bed	0.84	0.9	0.87
Chair	0.69	0.87	0.77
Desk	0.54	0.33	0.41
Dresser	0.74	0.76	0.75
Monitor	0.86	0.92	0.89
Nightstand	0.66	0.47	0.54
Sofa	0.81	0.91	0.85
Table	0.68	0.72	0.70
Toilet	0.81	0.88	0.84
Average	0.75	0.74	0.74

TABLE 2: Classification performance metrics per class.

C. Effect of Architectural Variants

To evaluate the influence of architectural choices, we tested the following variants:

- CNN only (Orion net without pretraining)
- CNN + contrastive learning
- Full Orion encoder + contrastive learning

Table 3 shows the classification accuracy for each variant. Pretraining with contrastive learning didn't improve accuracy over training from scratch.

In addition we found out that using as an encoder the full Orion network results in a lower contrastive loss as expected, despite this even if the transform head applied after the encoder is the same, the accuracy and classification loss has better results with the smaller encoder.

This is caused by the fact that, as shown in fig. 3, the major problem of the method shown is that, despite using dropout, it gravely overfits on the training data. Therefore using a bigger encoder this problem is amplified.

Model	Accuracy	Loss	NT-Xent loss
Full Orion	0.87	0.31	None
CNN			
+ contrastive learning	0.76	0.8	0.84
Full Orion encoder			
+ contrastive learning	0.70	0.93	0.36

TABLE 3: Classification accuracy for each tested model.

Table 4 shows the training time required for each model. Although the total time to train both the encoder and the transformation head is higher than training the Orion model alone, it's important to note that the encoder is trained only once. As a result, the transformation head can be retrained more quickly for new tasks. This suggests that, by reusing the encoder and retraining only the transformation head, our proposed model could offer faster training times when

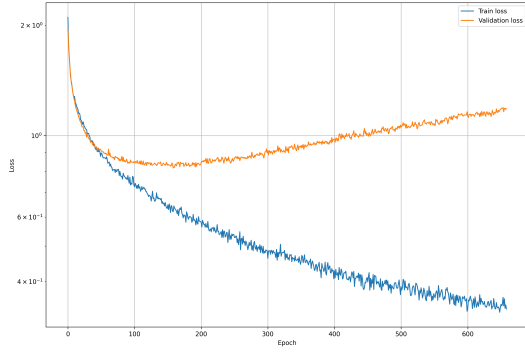


Fig. 3: Loss evolution during training of the classification transform head

adapting to different tasks.

The training times are found using a laptop with a NVIDIA RTX A3000 Laptop GPU and a 11th Gen Intel Core i9-11950H x16.

Model	Encoder train time (s)	Model train time (s)	Total (s)
Full Orion	0	2531	2531
CNN + contrastive learning	5950	735	6685
Full Orion encoder + contrastive learning	11350	700	12050

TABLE 4: Training time for each tested model.

D. Confusion Matrix

Finally, we present the confusion matrix for the classification model in Figure 4. It highlights class-wise prediction trends and misclassification patterns.

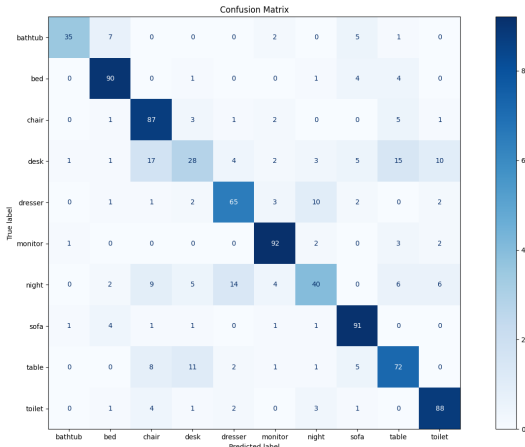


Fig. 4: Confusion matrix of the classification model on ModelNet10.

VI. CONCLUSION AND FUTURE WORK

In this project, we developed a modular learning framework for 3D object classification by combining ORION’s pose-aware encoder with contrastive learning and a projection head trained via the NT-Xent loss. The final classification head was trained in a supervised manner, enabling the evaluation of learned embeddings on the ModelNet10 benchmark.

Our results indicate that while the use of contrastive learning leads to a well-structured embedding space, it does not necessarily improve classification accuracy compared to training the model from scratch. Moreover, using the full ORION encoder for contrastive learning yielded lower contrastive loss but suffered from significant overfitting during classification. In contrast, a smaller CNN encoder achieved better generalization and higher classification accuracy. Training time analysis further revealed that, although the total time for contrastive learning is higher, the modular structure allows the encoder to be reused, making future adaptations faster and more scalable.

Future Work

Future work could explore the following directions:

- Investigating regularization strategies (e.g., stronger dropout, weight decay, or data augmentation) to mitigate overfitting during classification.
- Applying the framework to larger and more diverse datasets like ModelNet40 or ShapeNet to assess scalability.
- Exploring alternative projection head architectures to test other 3D objects tasks (like orientation classification) to test adaptability of the found encoder.
- Tuning of hyperparameters to reach higher accuracy.

Lessons Learned

Throughout the project, I’ve learned how to implement a contrastive learning framework and how 3D objects are represented in these classification tasks. A key challenge was for sure stabilizing the contrastive training phase and preventing overfitting, particularly when using deeper encoder architectures.

REFERENCES

- [1] D. Maturana and S. Scherer, “Voxnet: A 3d convolutional neural network for real-time object recognition,” in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 922–928, 2015.
- [2] Z. Wu, S. Song, A. Khosla, X. Tang, and J. Xiao, “3d shapenets for 2.5d object recognition and next-best-view prediction,” *CoRR*, vol. abs/1406.5670, 2014.
- [3] N. Sedaghat, M. Zolfaghari, and T. Brox, “Orientation-boosted voxel nets for 3d object recognition,” *CoRR*, vol. abs/1604.03351, 2016.
- [4] T. Chen, S. Kornblith, M. Norouzi, and G. E. Hinton, “A simple framework for contrastive learning of visual representations,” *CoRR*, vol. abs/2002.05709, 2020.
- [5] D. Nazir, M. Z. Afzal, A. Pagani, M. Liwicki, and D. Stricker, “Contrastive learning for 3d point clouds classification and shape completion,” *Sensors*, vol. 21, no. 21, 2021.
- [6] J. Jiang, X. Lu, W. Ouyang, and M. Wang, “Unsupervised representation learning for 3d point cloud data,” *CoRR*, vol. abs/2110.06632, 2021.
- [7] X. Wang and L. Aitchison, “How to set adam’s weight decay as you scale model and dataset size,” 2025.
- [8] I. Loshchilov and F. Hutter, “SGDR: stochastic gradient descent with restarts,” *CoRR*, vol. abs/1608.03983, 2016.

- [9] Z. Wu, S. Song, A. Khosla, X. Tang, and J. Xiao, “3d shapenets for 2.5d object recognition and next-best-view prediction,” *CoRR*, vol. abs/1406.5670, 2014.